

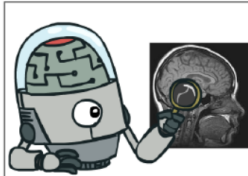
Course Info & Uninformed Search

David Izhaki

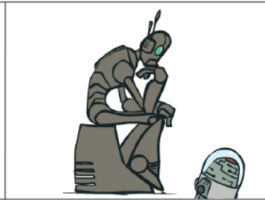
Department of Computer Science, Bar-Ilan

The science of making machines that:

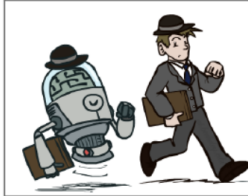
Think like people



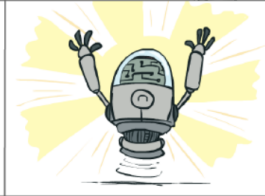
Think rationally



Act like people



Act rationally



- ▶ **Course Info & Logistics**
- ▶ State Space Basics
- ▶ Uninformed Search Algorithms

About the Course

- ▶ This course introduces the fundamentals of Artificial Intelligence.
- ▶ Introductory-level course: main areas and concepts that form the foundation of AI.
- ▶ Topics include:
 - Planning
 - Constraint Satisfaction Problems (CSP)
 - Markov Decision Processes (MDP)
 - Multi-Armed Bandits (MAB)
 - Reinforcement Learning (RL)
 - Games
 - MultiAgents

David Izhaki

Office hours: by prior arrangement via email

Email: davidizhakinew@gmail.com

- ▶ Course content will be uploaded to Moodle & Git.
- ▶ Exercises:
 - 3 exercises.
 - Individual work only, submissions must be done by each student separately.
 - All exercises must be written in Python, must run without errors, and within a reasonable time.
 - Plagiarism or copying will be handled severely!
 - The assignments will be published on Git.

Extensions Policy

- ▶ There are no grace days.
- ▶ If you need an extension, email the TA responsible for the exercise.
 - Sending an email does not guarantee that an extension will be approved.
- ▶ Late submission penalties:
 - 10% penalty for the first day late
 - additional 20% for the second day late
 - After that: automatic grade of 0
- ▶ No need to notify us in advance (unless you request an extension).

Outline

- ▶ ~~Course Info & Logistics~~
- ▶ **State Space Basics**
- ▶ Uninformed Search Algorithms

Search Problems in State Space

- ▶ Today we will work on **search problems**.
- ▶ In many CS courses you already met **graph search**:
 - You were given a graph $G = (V, E)$, and sometimes also a weight function $w : E \rightarrow \mathbb{R}$.
 - Your job was to run BFS / DFS / Dijkstra to find a (shortest) path between two vertices.

Search Problems in State Space

- ▶ Today we will work on **search problems**.
- ▶ In many CS courses you already met **graph search**:
 - You were given a graph $G = (V, E)$, and sometimes also a weight function $w : E \rightarrow \mathbb{R}$.
 - Your job was to run BFS / DFS / Dijkstra to find a (shortest) path between two vertices.
- ▶ In AI, the story is a bit different: **nobody gives us the graph**.
 - Why? It is usually **too large** to fit in the computer's memory.
 - The vertices are **states**, which describe situations of the world.
 - The edges are **actions**: we create them ourselves by applying an action to a state.
 - So the graph is **implicit**, and we discover it while we search.

State Space

Definition

- ▶ A **state** is a description of the relevant facts about the world.
- ▶ The **state space** is the set of all possible states.

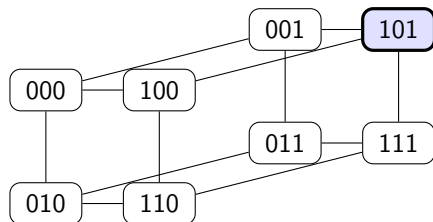
Formally: we describe the world using a set of **variables** $\mathcal{V} = \{v_1, \dots, v_n\}$, where each variable v_i has a **domain** $D(v_i)$ of possible values.

- ▶ A **state** s assigns a value to every variable: $s[v_i] \in D(v_i)$ for every i .
- ▶ The **state space** is the set of all such assignments:

$$S = D(v_1) \times D(v_2) \times \dots \times D(v_n), \quad |S| = \prod_{i=1}^n |D(v_i)|$$

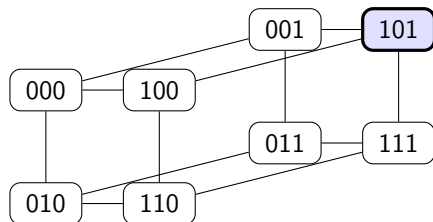
Example: State Space with 3 Students

- ▶ We have 3 students A, B, C . Each one is either at the university (1) or not (0).
- ▶ Formally: variables $\mathcal{V} = \{A, B, C\}$, each with domain $\{0, 1\}$.
- ▶ A state is a 3-bit vector, e.g. $s = (1, 0, 1)$: A and C are at the university, B is not.
- ▶ Number of states: $|S| = 2^3 = 8$.
- ▶ An action changes the bit of one student ($TurnOn(X)$ / $TurnOff(X)$), so every state has exactly **3 neighbors**.



Example: State Space with 3 Students

- ▶ We have 3 students A, B, C . Each one is either at the university (1) or not (0).
- ▶ Formally: variables $\mathcal{V} = \{A, B, C\}$, each with domain $\{0, 1\}$.
- ▶ A state is a 3-bit vector, e.g. $s = (1, 0, 1)$: A and C are at the university, B is not.
- ▶ Number of states: $|S| = 2^3 = 8$.
- ▶ An action changes the bit of one student ($TurnOn(X)$ / $TurnOff(X)$), so every state has exactly **3 neighbors**.



But what if we had 100 students? Then $|S| = 2^{100} \approx 10^{30}$ states.

No computer can store this graph, so we never build it.

Search in the State Space

- ▶ So how do we solve such problems in AI?
We define them as a **search problem in the state space**.
- ▶ **Today: uninformed search.** You already know these algorithms from other courses (BFS, DFS, ...). Today we will see how to use them when the graph is implicit.
- ▶ **Practice 2: heuristics.** We will add knowledge about the problem and turn our algorithms into **informed search**.
- ▶ **Practice 3: planning.** A formal language for describing search problems.

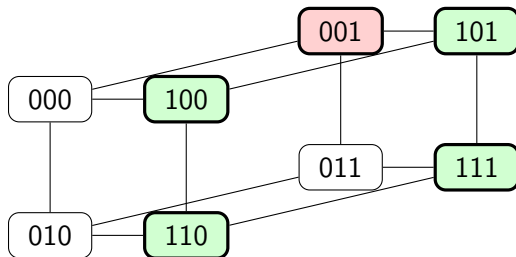
Search Problem: Definition

A search problem in the state space is defined by:

- ① An **initial state** I : where we start.
- ② A set of **actions** that can be applied in a state.
- ③ A **transition function** $T(s, a) = s'$: the state we reach after applying action a in state s .
- ④ A **goal condition** G : a state s is a **goal state** if it satisfies G .
- ⑤ (Optional) a **cost** for each action.

Back to Our Example: Start and Goal

- ▶ Start: $I = 001$ (red).
- ▶ Goal: $G =$ “A is at the university”, i.e. the first bit is 1.
- ▶ Notice: **many states satisfy** G . Every state $1xy$ is a goal state, so there are $2^2 = 4$ goal states (green).



What Do We Want to Find?

Given a search problem, these are the typical questions:

- 1 Does there exist a path from l to **some** goal state?
- 2 Find **a** path from l to **some** goal state.
- 3 Find the **optimal** (cheapest) path from l to **some** goal state.

Guiding Questions

- 1 Have we clearly stated the **objective / goal test**?
- 2 Is the **state representation** precise and complete?
- 3 Did we list all **possible actions**?
- 4 Do actions ever produce **illegal states**?
- 5 Are transitions **deterministic** or **stochastic** (we will discuss this later in the course)?

What Makes an Algorithm “Good”?

- ▶ In algorithms courses, we usually look for **the best** algorithm for a problem, with a guarantee on its worst-case running time.
- ▶ In AI the story is different: many of our problems are **computationally hard**. In the worst case, every known algorithm takes exponential time (remember the 2^n states).
- ▶ Also, there is **no single measure** of quality: one algorithm is faster, another uses less memory, a third finds cheaper solutions.
- ▶ So we look for **good trade-offs** between the **resources** we use (time, memory) and the **quality of the solution** (e.g. its cost), and we prefer methods that work well **in practice**.

Search Algorithms: Key Concepts

Completeness and Optimality

- ▶ An algorithm is **complete** if:
 - it finds a solution whenever a solution exists, and
 - if no solution exists, it terminates and reports **no solution**.
- ▶ An algorithm is **optimal** if, whenever a solution exists, it returns a solution with **minimum cost**.

Search Algorithms: Key Concepts

Systematic Search vs. Local Search

► Systematic search:

- Explores the state space **globally**, in an organized way.
- Keeps many discovered states in a data structure (the **frontier**) and expands the search outward across the graph.

► Local search:

- Keeps only **one (or a few) current states** at a time.
- Moves from the current state to a neighbor, without trying to explore the whole graph.

- ▶ ~~Course Info & Logistics~~
- ▶ ~~State Space Basics~~
- ▶ **Uninformed Search Algorithms**
 - 1 Systematic search algorithms
 - 2 Local search algorithms

Back to Our Plan: Uninformed Search

- ▶ Remember, today we focus on **uninformed (blind) search**: the algorithm knows **nothing** about the problem beyond its definition, so it uses **no heuristics**.
- ▶ These algorithms are usually **easy to implement**, but often **inefficient** on large state spaces.
- ▶ We will go over two families:
 - **Systematic search**: BFS, DFS, DFS-L, IDS, UCS.
 - **Local search**: Random Walk.

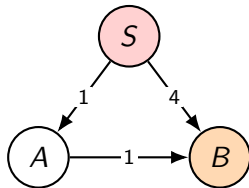
What Do We Store During the Search?

What you already know

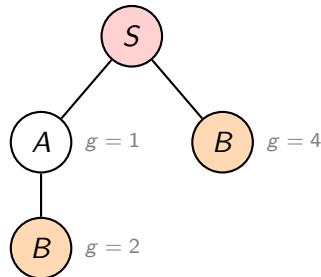
- ▶ In BFS from the algorithms course, you stored information in **arrays indexed by vertices**: `visited[v]`, `parent[v]`, `dist[v]`.
- ▶ In AI we cannot do this: the graph is implicit, so we **do not have a list of vertices**.
- ▶ Also, we need to return the **plan**: the sequence of actions that leads to a goal.
- ▶ So every time we generate a state, we create an object that carries this information itself. We call it a **node**.

What Do We Store During the Search?

States vs. Nodes



State-space graph:
each state appears **once**.



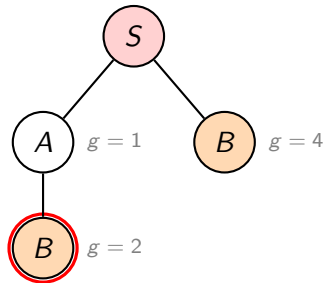
Search tree:
each node is a **path** from I .
One state (B), **two nodes**.

What Do We Store During the Search?

What a node stores

- ① **state**: which state of the graph we reached. e.g. B
- ② **parent**: the node that generated this node. e.g. the node of A
- ③ **action**: the action applied at the parent. e.g. the action from A to B
- ④ **g** : the cost of the path from I to this node.

Following the parents back to the root gives us the **plan**.



Search tree:

each node is a **path** from I .
One state (B), **two nodes**.

Helper Procedures

How we build the search tree

All our search algorithms build the search tree in the same way, using four helper procedures:

- ▶ We start from I and create the **root node**: `make_root_node`.
- ▶ We ask which states we can reach from a state: `successors`.
- ▶ For each successor, we create a **child node**: `make_node`.
- ▶ When we reach a goal, we follow the parents back to the root to get the **plan**: `extract_solution`.

Helper Procedures

make_root_node: where we start

make_root_node

```
1 function make_root_node(s):  
2    $\sigma \leftarrow$  new node  
3   state( $\sigma$ )  $\leftarrow$  s  
4   parent( $\sigma$ )  $\leftarrow$  undefined  
5   action( $\sigma$ )  $\leftarrow$  undefined  
6   g( $\sigma$ )  $\leftarrow$  0  
7   return  $\sigma$ 
```



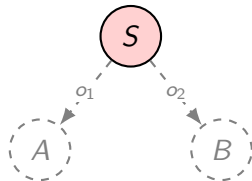
state: S
parent: undefined
action: undefined
g: 0

Helper Procedures

successors: where can we go?

successors

```
1 function successors(s):  
2     result  $\leftarrow \emptyset$   
3     for each action o:  
4         if applicable(o,s):  
5              $s' \leftarrow \text{apply}(o,s)$   
6             result  $\leftarrow \text{result} \cup \{\langle o, s' \rangle\}$   
7     return result
```



$$\text{successors}(S) = \{\langle o_1, A \rangle, \langle o_2, B \rangle\}$$

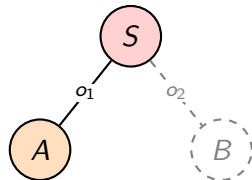
Later we will see that generating *all* successors is not always a good idea: in Lec 3 and HW1 we will see pruning techniques.

Helper Procedures

make_node: creating a child

make_node

```
1 function make_node( $\sigma, o, s$ ):  
2    $\sigma' \leftarrow$  new node  
3   state( $\sigma'$ )  $\leftarrow s$   
4   parent( $\sigma'$ )  $\leftarrow \sigma$  // who generated it  
5   // action applied at the parent  
6   action( $\sigma'$ )  $\leftarrow o$   
7    $g(\sigma') \leftarrow g(\sigma) + \text{cost}(o)$   
8   return  $\sigma'$ 
```



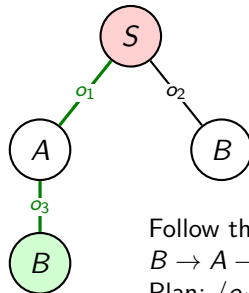
state: A
parent: node of S
action: o_1
 g : $0 + 1 = 1$

Helper Procedures

extract_solution: reading the plan

extract_solution

```
1 function extract_solution( $\sigma$ ):  
2   solution  $\leftarrow$  new stack  
3   while parent( $\sigma$ ) is defined:  
4     // backwards pushing  
5     solution.push(action( $\sigma$ ))  
6      $\sigma \leftarrow$  parent( $\sigma$ )  
7   return solution  
8   // popping gives I  $\rightarrow$  goal
```



Follow the parents:
 $B \rightarrow A \rightarrow S$
Plan: $\langle o_1, o_3 \rangle$

- ▶ ~~Course Info & Logistics~~
- ▶ ~~State Space Basics~~
- ▶ **Uninformed Search Algorithms**
 - ① **Systematic search algorithms**
 - **BFS (Breadth-First Search)**
 - DFS (Depth-First Search)
 - Depth-Limited DFS (DFS-L)
 - Iterative Deepening DFS (IDDFS)
 - UCS (Uniform-Cost Search)
 - ② Local search algorithms

BFS (Breadth-First Search)

BFS — Pseudocode

```
1  function BFS( $s_0$ ):
2      if is_goal( $s_0$ ):
3          return extract_solution(make_root_node( $s_0$ ))
4      queue  $\leftarrow$  new fifo-queue
5      queue.push(make_root_node( $s_0$ ))
6
7      while not queue.empty():
8           $\sigma \leftarrow$  queue.pop()
9          for each  $\langle o, s \rangle$  in successors(state( $\sigma$ )):
10              $\sigma' \leftarrow$  make_node( $\sigma, o, s$ )
11             if is_goal(state( $\sigma'$ )):
12                 return extract_solution( $\sigma'$ )
13
14             queue.push( $\sigma'$ )
15  return unsolvable
```

BFS (Breadth-First Search)

BFS — Pseudocode

```
1  function BFS( $s_0$ ):
2      if is_goal( $s_0$ ):
3          return extract_solution(make_root_node( $s_0$ ))
4      queue  $\leftarrow$  new fifo-queue
5      queue.push(make_root_node( $s_0$ ))
6
7      while not queue.empty():
8           $\sigma \leftarrow$  queue.pop()
9          for each  $\langle o, s \rangle$  in successors(state( $\sigma$ )):
10              $\sigma' \leftarrow$  make_node( $\sigma, o, s$ )
11             if is_goal(state( $\sigma'$ )): 
12                 return extract_solution( $\sigma'$ )
13
14             queue.push( $\sigma'$ )
15  return unsolvable
```

Question:

What's the difference between this pseudocode and the lecture's pseudocode?

BFS (Breadth-First Search)

BFS — Pseudocode

```
1  function BFS( $s_0$ ):
2      if is_goal( $s_0$ ):
3          return extract_solution(make_root_node( $s_0$ ))
4      queue  $\leftarrow$  new fifo-queue
5      queue.push(make_root_node( $s_0$ ))
6
7      while not queue.empty():
8           $\sigma \leftarrow$  queue.pop()
9          for each  $\langle o, s \rangle$  in successors(state( $\sigma$ )):
10              $\sigma' \leftarrow$  make_node( $\sigma, o, s$ )
11             if is_goal(state( $\sigma'$ )):  $\leftarrow$ 
12                 return extract_solution( $\sigma'$ )
13
14             queue.push( $\sigma'$ )
15  return unsolvable
```

Answer:

Here we test the goal *when generating successors*, so we can stop as soon as a goal is discovered instead of waiting until it is popped from the queue.

This does *not* hurt BFS optimality (with uniform step costs), because BFS still explores in non-decreasing depth.

Let's Run BFS!

- ▶ Enough theory, let's see BFS in action in the simulator.
- ▶ While we run it, pay attention to:
 - The **frontier**: a FIFO queue, so the oldest node is expanded first.
 - The **search tree**: it grows level by level.
 - **States vs. nodes**: the same state can appear in several nodes.
 - The line of the **pseudocode** that runs in each step.
- ▶ The simulator is on our GitHub, so you can play with it at home on your own graphs.

BFS with Duplicate Detection

BFS with CLOSED

```
1  function BFS( $s_0$ ):
2      if is_goal( $s_0$ ):
3          return extract_solution(make_root_node( $s_0$ ))
4      queue  $\leftarrow$  new FIFO-queue
5      queue.push(make_root_node( $s_0$ ))
6      closed  $\leftarrow \emptyset$ 
7      while not queue.empty():
8           $\sigma \leftarrow$  queue.pop()
9          if state( $\sigma$ )  $\notin$  closed:
10             closed  $\leftarrow$  closed  $\cup$  {state( $\sigma$ )}
11             for each  $\langle o, s \rangle$  in successors(state( $\sigma$ )):
12                  $\sigma' \leftarrow$  make_node( $\sigma, o, s$ )
13                 if is_goal(state( $\sigma'$ )):
14                     return extract_solution( $\sigma'$ )
15                 queue.push( $\sigma'$ )
16      return unsolvable
```

BFS: Analysis

Completeness and Optimality

Completeness:

- ▶ **Without CLOSED: no.** If there is no solution and the graph has a cycle, BFS runs forever. (If a solution exists, BFS does find it.)
- ▶ **With CLOSED: yes**, on a finite state space. Every state is expanded at most once, so BFS stops and reports *unsolvable*.

Optimality (with or without CLOSED):

- ▶ **Unit costs: yes.** BFS generates nodes level by level, so the first goal it finds is the **shallowest** one.
- ▶ **General costs: no.** The shallowest solution is not always the cheapest.

BFS: Analysis

Complexity

Notation: b = branching factor (max number of successors of a state), d = depth of the shallowest goal.

Time (nodes generated). In the worst case we generate every node up to depth d :

$$1 + b + b^2 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1} = O(b^d)$$

We test the goal when a node is *generated*, so we never generate depth $d + 1$.

Space: the queue may hold a whole level of the tree: $O(b^d)$.

- ▶ ~~Course Info & Logistics~~
- ▶ ~~State Space Basics~~
- ▶ **Uninformed Search Algorithms**
 - ① **Systematic search algorithms**
 - ~~BFS (Breadth-First Search)~~
 - **DFS (Depth-First Search)**
 - Depth-Limited DFS (DFS-L)
 - Iterative Deepening DFS (IDDFS)
 - UCS (Uniform-Cost Search)
 - ② Local search algorithms

DFS (Depth-First Search)

DFS

```
1  function DFS( $s_0$ ):
2      if is_goal( $s_0$ ):
3          return extract_solution(make_root_node( $s_0$ ))
4      stack  $\leftarrow$  new LIFO-stack
5      stack.push(make_root_node( $s_0$ ))
6      closed  $\leftarrow \emptyset$ 
7      while not stack.empty():
8           $\sigma \leftarrow$  stack.pop()
9          if state( $\sigma$ )  $\notin$  closed:
10             closed  $\leftarrow$  closed  $\cup$  {state( $\sigma$ )}
11             for each  $\langle o, s \rangle$  in successors(state( $\sigma$ )):
12                  $\sigma' \leftarrow$  make_node( $\sigma, o, s$ )
13                 if is_goal(state( $\sigma'$ )):
14                     return extract_solution( $\sigma'$ )
15                 stack.push( $\sigma'$ )
16      return unsolvable
```

DFS: Analysis

Completeness and Optimality

Completeness:

- ▶ **Without CLOSED: no.** DFS can get stuck in a cycle (or go down an infinite path) and never come back, **even when a solution exists.**
- ▶ **With CLOSED: yes,** on a finite state space.

Optimality: no (with or without CLOSED).

- ▶ DFS returns the first goal it reaches, which can be deep and expensive.

DFS: Analysis

Complexity

Notation: b , d as before, and m = the maximum depth of the state space (can be much larger than d , or even infinite).

Time: in the worst case DFS explores the whole tree down to depth m :

$$1 + b + b^2 + \dots + b^m = O(b^m)$$

Space: only the current path, plus up to b siblings per level: $O(b \cdot m)$. Much smaller than the $O(b^d)$ of BFS!

- ▶ ~~Course Info & Logistics~~
- ▶ ~~State Space Basics~~
- ▶ **Uninformed Search Algorithms**
 - ① **Systematic search algorithms**
 - ~~BFS (Breadth-First Search)~~
 - ~~DFS (Depth-First Search)~~
 - **Depth-Limited DFS (DFS-L)**
 - Iterative Deepening DFS (IDDFS)
 - UCS (Uniform-Cost Search)
 - ② Local search algorithms

DFS-L (Depth-Limited DFS)

DFS-L

```
1  function DFS_L( $s_0$ , limit):
2      stack  $\leftarrow$  new LIFO-stack
3      stack.push( $\langle$ make_root_node( $s_0$ ), 0 $\rangle$ )           // (node, depth)
4      cutoff  $\leftarrow$  false
5      closed  $\leftarrow \emptyset$                          // state  $\rightarrow$  depth
6      while not stack.empty():
7           $\langle \sigma, d \rangle \leftarrow$  stack.pop()
8          if is_goal(state( $\sigma$ )):
9              return extract_solution( $\sigma$ )
10         if state( $\sigma$ )  $\notin$  closed or closed[state( $\sigma$ )]  $> d$ :
11             closed[state( $\sigma$ )]  $\leftarrow d$ 
12             if  $d < \text{limit}$ :
13                 for each  $\langle o, s \rangle$  in successors(state( $\sigma$ )):
14                     stack.push( $\langle$ make_node( $\sigma, o, s$ ),  $d + 1$  $\rangle$ )
15             else if successors(state( $\sigma$ ))  $\neq \emptyset$ :
16                 cutoff  $\leftarrow$  true                 // something was left deeper
17     return CUTOFF if cutoff else UNSOLVABLE
```

DFS-L: Completeness, Optimality & Complexity

Definitions:

- ▶ Branching factor: b = maximum number of successors generated when expanding a state.
- ▶ Goal depth: d .
- ▶ Max depth of the search space: m (could be large or infinite); unit step costs unless noted.
- ▶ Depth limit: L (DFS-L does not expand beyond depth L).

Completeness: No!

- ▶ If $L < d$: **not complete** $\rightarrow$ may return **cutoff** (no solution within the limit).
- ▶ If $L \geq d$ and $b < \infty$: **complete** (a goal at depth d will be found).

Optimality:

- ▶ **No.** With $L > d$, it's act like DFS and can find a solution that bigger then d .

DFS-L: Completeness, Optimality & Complexity

Time (nodes generated, worst case up to the limit):

$$N(L) = \sum_{i=0}^L b^i = \frac{b^{L+1} - 1}{b - 1} = \Theta(b^L)$$

Space (memory usage): $\Theta(b \cdot L)$

(Stores only the current path plus up to b siblings per level; with CLOSED there is extra bookkeeping, but still linear in L for typical analyses.)

- ▶ ~~Course Info & Logistics~~
- ▶ ~~State Space Basics~~
- ▶ **Uninformed Search Algorithms**
 - ① **Systematic search algorithms**
 - ~~BFS (Breadth-First Search)~~
 - ~~DFS (Depth-First Search)~~
 - ~~Depth-Limited DFS (DFS-L)~~
 - **Iterative Deepening DFS (IDDFS)**
 - UCS (Uniform-Cost Search)
 - ② Local search algorithms

IDS (Iterative Deepening Search)

IDS — Pseudocode

```
1  function IDS( $s_0$ , max_depth =  $\infty$ ):
2      limit  $\leftarrow$  0
3      while limit  $\leq$  max_depth:
4           $\langle$ status, path $\rangle \leftarrow$  DFS_L_status( $s_0$ , limit)
5
6          if status = FOUND:
7              return path
8          if status = unsolvable:
9              return unsolvable
10         // status = CUTOFF: goal may exist deeper than current limit
11         limit  $\leftarrow$  limit + 1
```

IDS: Completeness, Optimality & Complexity

Definitions:

- ▶ Branching factor: b = maximum number of successors produced when expanding a state.
- ▶ Goal depth: d
- ▶ Max depth of the search space: m (could be large or infinite); unit step costs unless noted.

Completeness:

- ▶ Yes (for finite b and with **duplicate detection**).
IDS will eventually try $L = d$ and find the goal.

Optimality (unit costs):

- ▶ Yes. The first time a goal is reached is at the **minimum depth**.

IDS: Completeness, Optimality & Complexity

Time (nodes generated, worst case up to the limit):

- ▶ IDS runs DFS-L with $L = 0, 1, \dots, d$. Nodes at depth i are regenerated for each $L > i$, i.e., $d - i + 1$ times.

$$N = \sum_{L=0}^d \sum_{i=0}^L b^i = \sum_{i=0}^d (d - i + 1) b^i$$

- ▶ **Asymptotically (for $b > 1$):** $N = \Theta(b^d)$

Space (memory usage):

$$\Theta(b \cdot d)$$

Key takeaways

- ▶ Same asymptotic time as BFS: $\Theta(b^d)$, but with DFS-like memory: $\Theta(bd)$.
- ▶ Complete & optimal (unit cost) without storing the entire frontier.
- ▶ Great default when you don't know d and memory is tight.

- ▶ ~~Course Info & Logistics~~
- ▶ ~~State Space Basics~~
- ▶ **Uninformed Search Algorithms**
 - ① **Systematic search algorithms**
 - ~~BFS (Breadth-First Search)~~
 - ~~DFS (Depth-First Search)~~
 - ~~Depth-Limited DFS (DFS-L)~~
 - ~~Iterative Deepening DFS (IDDFS)~~
 - **UCS (Uniform-Cost Search)**
 - ② Local search algorithms

UCS: Uniform-Cost Search

- ▶ Remember the simulator: on “*Same graph, different answers*”, BFS returned $S \rightarrow D \rightarrow G$ with cost 11, but $S \rightarrow A \rightarrow B \rightarrow G$ costs only 4.
- ▶ BFS counts **steps**, not **cost**. When actions have different (non-negative) costs, we need a different order.
- ▶ **Idea:** always expand the node with the **cheapest path so far**. The frontier is a **priority queue** ordered by $g(\sigma)$.
- ▶ Sounds familiar? This is **Dijkstra**, adapted to our setting:
 - the graph is implicit,
 - we stop as soon as a **goal is popped**,
 - and we simply push duplicates instead of updating keys.

UCS (Uniform-Cost Search)

UCS

```
1  function UCS( $s_0$ ):
2      OPEN  $\leftarrow$  priority_queue ordered by  $f(\sigma) = g(\sigma)$     // min-heap by path cost
3      CLOSED  $\leftarrow \emptyset$                                      // expanded states
4      OPEN.push(make_root_node( $s_0$ ))
5      while not OPEN.empty():
6           $\sigma \leftarrow$  OPEN.pop_min()                        // node with smallest g
7          if is_goal(state( $\sigma$ )):                             // goal test when POPPED
8              return extract_solution( $\sigma$ )
9          if state( $\sigma$ )  $\notin$  CLOSED:
10             CLOSED  $\leftarrow$  CLOSED  $\cup$  {state( $\sigma$ )}
11             for each  $\langle o, s \rangle$  in successors(state( $\sigma$ )):
12                  $\sigma' \leftarrow$  make_node( $\sigma, o, s$ )
13                 OPEN.push( $\sigma'$ )
14  return unsolvable
```

Let's Run UCS!

- ▶ Back to the simulator: “*UCS example (weighted)*”.
- ▶ While we run it, pay attention to:
 - **OPEN** is sorted by g , so the cheapest node is always popped next.
 - The goal G is **generated** with $g = 9$ before we find the path with $g = 4$. Why do we test the goal only when a node is **popped**?
- ▶ Then: “*Same graph, different answers*” → **Compare all**.

UCS: Analysis

Assume all action costs are **non-negative**.

Completeness:

- ▶ **Without CLOSED: no.** If there is no solution and the graph has a cycle, UCS runs forever.
- ▶ **With CLOSED: yes**, on a finite state space.

Optimality: yes. The **first goal popped** from OPEN has the minimum path cost.

With CLOSED, a state is expanded only once, the first time it is popped, and at that moment its g is already the cheapest.

Complexity: let C^* = the cost of the optimal solution and ϵ = the smallest action cost (> 0). Then

$$\text{Time, Space} = O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$$

See Russell & Norvig, *AI: A Modern Approach*, Ch. 3 (Uniform-cost search).

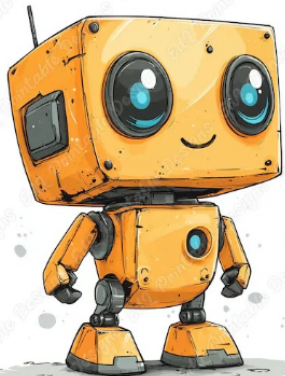
- ▶ ~~Course Info & Logistics~~
- ▶ ~~State Space Basics~~
- ▶ **Uninformed Search Algorithms**
 - ① ~~Systematic search algorithms~~
 - ② **Local search algorithms**
 - **Random walk**

Random Walk

Random Walk

```
1  $\sigma \leftarrow$  make-root-node(init())
2
3 forever:
4     if is-goal(state( $\sigma$ )):
5         return extract-solution( $\sigma$ )
6
7     choose a random element ( $o, s$ ) from succ(state( $\sigma$ ))
8      $\sigma \leftarrow$  make-node( $\sigma, o, s$ )
```

- ▶ The algorithm usually does not find a solution unless many action sequences are valid plans.
- ▶ It may run forever without making real progress.
- ▶ It can also fail at a dead end: a state with no successors.
- ▶ This is a common weakness of many local search methods.



Questions ?